SAMSUNG

Samsung Innovation Campus

Generative AI Module

Together for Tomorrow!
Enabling People
Education for Future Generations

Notas de Clase

Generative AI Module

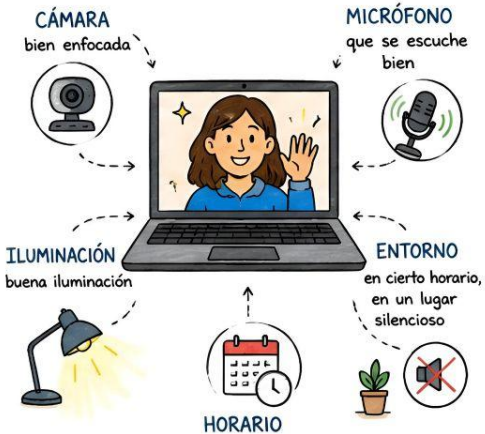
OBJETIVOS DEL CURSO

1 Se lleven como razonar un problema y saber cuando usar AI y cuando no.



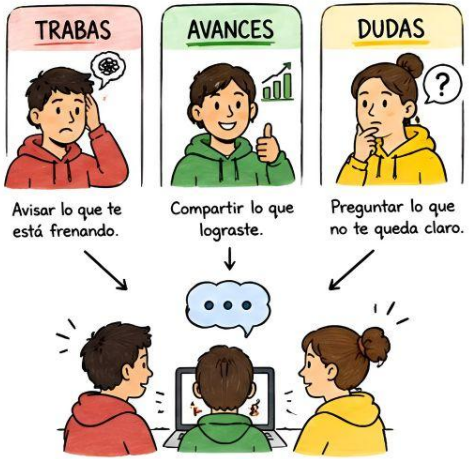
Primero pensar, después decidir.

2 Organizar su vida para tener meets, eso implica tener tu carta de presentacion en buenas condiciones:



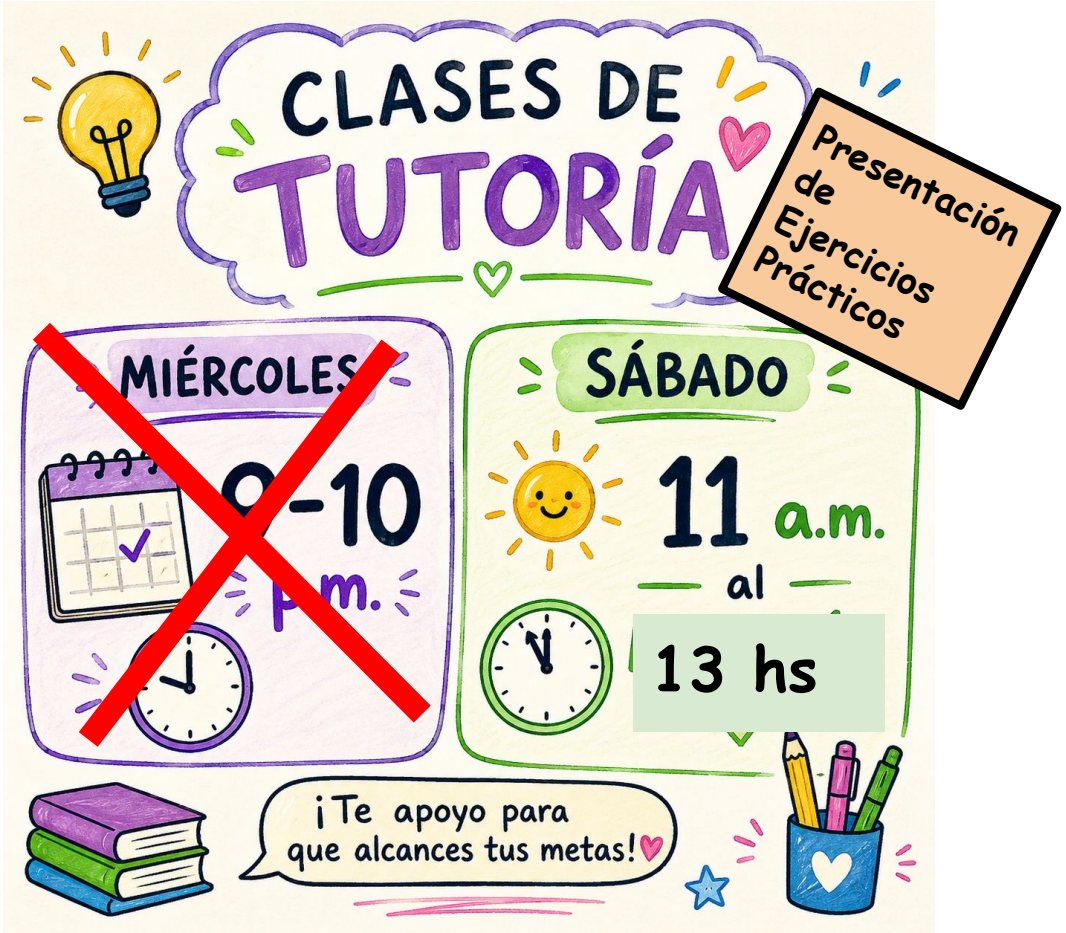
Tu meet es tu carta de presentación. ¡Cuidala!

3 Poder comunicar trabas, avances y dudas a tiempo, es preferible equivocarse aca que con una deadline.

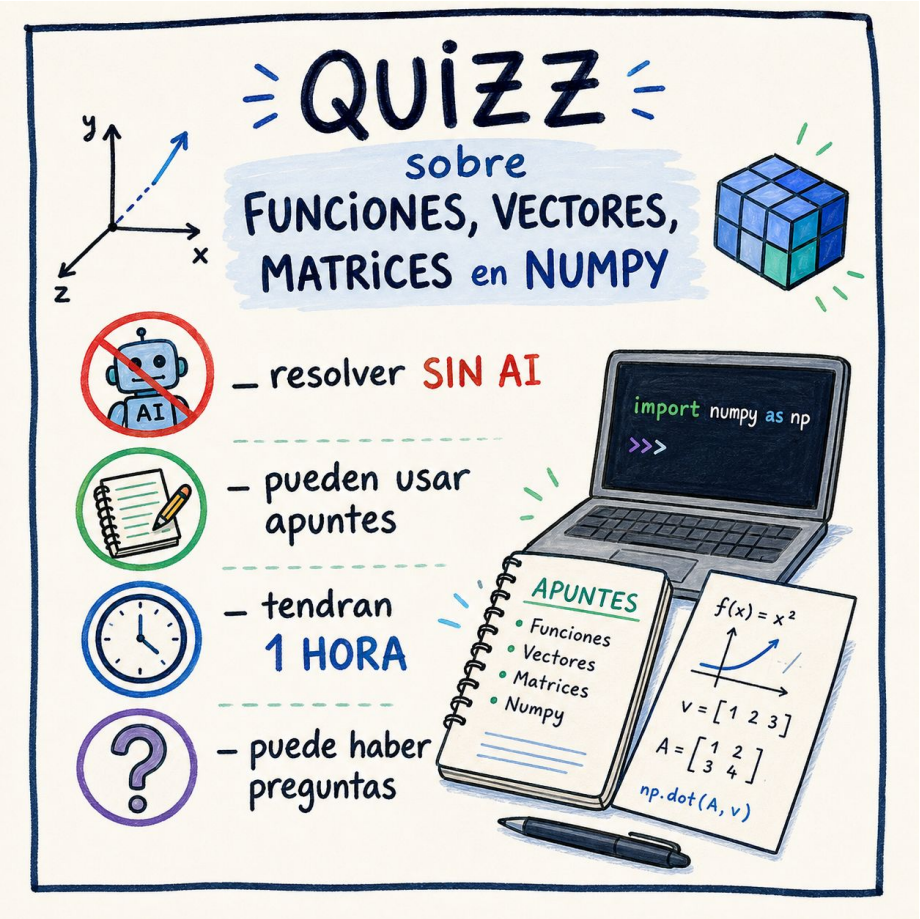


Comunicar a tiempo = mejores soluciones, menos estrés.

PENSAR. ORGANIZAR. COMUNICAR. = CRECER JUNTOS



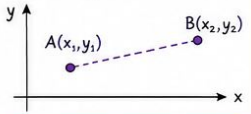
[Inscripción a la reunión - Zoom]


QUIZZ
sobre
FUNCIONES, VECTORES, MATRICES en NUMPY
y
x
z
AI
– resolver SIN AI
– pueden usar apuntes
– tendran 1 HORA
– puede haber preguntas
import numpy as np
>>>
APUNTES
• Funciones
• Vectores
• Matrices
• Numpy
f(x) = x²
v = [1 2 3]
A = [1 2 / 3 4]
np.dot(A, v)

Recordatorio …

1. DISTANCIA EUCLIDIANA

Mide la distancia en línea recta entre dos puntos o vectores.



$$d(\bar{a},\bar{b}) = \sqrt{\sum_{i=1}^{n}(a_i - b_i)^2}$$

En 2D:

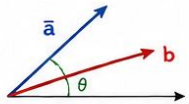
$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

NumPy:

```
np.linalg.norm(a - b)
```

2. DISTANCIA COSENO

Mide qué tan parecidos son dos vectores según su dirección.



$$\text{Similitud Coseno} = \frac{\bar{a}\cdot\bar{b}}{\|\bar{a}\|\,\|\bar{b}\|}$$

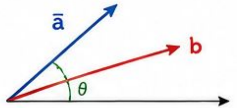
$$\text{Distancia Coseno} = 1 - \frac{\bar{a}\cdot\bar{b}}{\|\bar{a}\|\,\|\bar{b}\|}$$

NumPy:

```
cos_sim = np.dot(a,b)/(np.linalg.norm(a)*np.linalg.norm(b))
cos_dist = 1 - cos_sim
```

3. PRODUCTO PUNTO (DOT PRODUCT)

Resultado escalar.



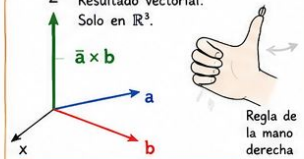
$$\bar{a}\cdot\bar{b} = \sum_{i=1}^{n} a_i b_i = \|\bar{a}\|\,\|\bar{b}\|\cos\theta$$

NumPy:

```
np.dot(a,b)  o  a @ b
```

4. PRODUCTO VECTORIAL (CROSS PRODUCT)

Resultado vectorial. Solo en $\mathbb{R}^3$.



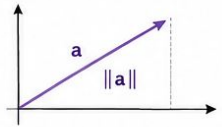
$$\bar{a}\times\bar{b} = \begin{bmatrix} a_2b_3 - a_3b_2 \\ a_3b_1 - a_1b_3 \\ a_1b_2 - a_2b_1 \end{bmatrix}$$

NumPy:

```
np.cross(a,b)
```

5. NORMA DE UN VECTOR

Longitud o magnitud del vector.



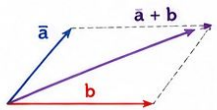
$$\|\mathbf{a}\| = \sqrt{\sum_{i=1}^{n} a_i^2}$$

NumPy:

```
np.linalg.norm(a)
```

6. SUMA DE VECTORES

Se suman componente a componente.



$$\bar{a}+\bar{b} = (a_1+b_1,\ a_2+b_2,\ \ldots,\ a_n+b_n)$$

NumPy:

```
a + b
```

7. DETERMINANTE

Para matriz 2×2:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$\det(A) = ad - bc$$

Para matriz 3×3:

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

$$\det(A) = aei + bfg + cdh - ceg - bdi - afh$$

NumPy:

```
np.linalg.det(A)
```

8. DERIVADAS (SYMPY)

Primer orden, una variable.

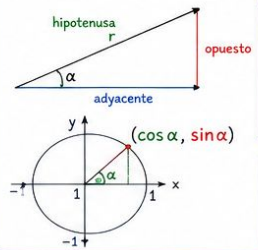
Sea $f(x) = x^3 + 2x^2 - 5x + 1$

$f'(x) = 3x^2 + 4x - 5$

SymPy:

```
import sympy as sp
x = sp.symbols('x')
f = x**3 + 2*x**2 - 5*x + 1
sp.diff(f, x)  # primera derivada
```

9. FUNCIONES TRIGONOMÉTRICAS



$$\sin(\alpha) = \frac{\text{opuesto}}{\text{hipotenusa}}$$

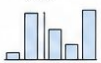
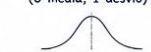
$$\cos(\alpha) = \frac{\text{adyacente}}{\text{hipotenusa}}$$

$$\tan(\alpha) = \frac{\text{opuesto}}{\text{adyacente}}$$

NumPy:

```
np.sin(x)
np.cos(x)
np.tan(x)
# x en radianes
```

10. FUNCIONES RANDOM DE NUMPY

Número aleatorio [0,1)	Vector aleatorio [0,1)	Matriz aleatoria [0,1)	Enteros aleatorios en [bajo, alto)	Distribución Normal (0 media, 1 desvío)	Elegir elementos al azar	Mezclar un vector (en el lugar)	Fijar semilla (reproducibilidad)
							
`np.random.rand()`	`np.random.rand(5)`	`np.random.rand(3,3)`	`np.random.randint(0,10,size=5)`	`np.random.randn(5)` # o `np.random.normal(loc=0, scale=1, size=5)`	`np.random.choice([1,2,3,4])`	`v = np.arange(10)` `np.random.shuffle(v)`	`np.random.seed(42)`

*args y **kwargs en Python

***args** → Cantidad variable de argumentos posicionales

Se almacenan como una TUPLA.

```python
def sumar(*args):
    print(args)
    print(type(args))
    return sum(args)


print(sumar(10, 20, 30, 40))
```

Salida

```
(10, 20, 30, 40)
<class 'tuple'>
100
```

TUPLA: (10, 20, 30, 40)

¿Cuándo usarlo?

- No sabes cuántos parámetros recibirá la función.
- Todos llegan por POSICIÓN.

****kwargs** → Cantidad variable de argumentos con nombre

Se almacenan como un DICCIONARIO.

```python
def mostrar(**kwargs):
    print(kwargs)
    print(type(kwargs))

mostrar(
    nombre="Ana",
    edad=25,
    ciudad="Buenos Aires"
)
```

Salida

```
{'nombre': 'Ana', 'edad': 25,
 'ciudad': 'Buenos Aires'}
<class 'dict'>
```

DICCIONARIO

nombre	'Ana'
edad	25
ciudad	'Buenos Aires'

¿Cuándo usarlo?

- No sabes qué parámetros llegarán.
- Cada argumento tiene un NOMBRE (clave).

Ambos juntos

```python
def informacion(nombre, *args, **kwargs):
    print("Nombre:", nombre)
    print("Args:", args)
    print("Kwargs:", kwargs)

informacion(
    "Carlos",
    10, 20, 30,
    edad=28,
    ciudad="Lima"
)
```

Resultado

```
Nombre: Carlos
Args: (10, 20, 30)
Kwargs: {'edad': 28, 'ciudad': 'Lima'}
```

Orden correcto de los parámetros

```python
def funcion(
    parametro,
    *args,
    **kwargs
):
    pass
```

- parametro → Parámetro normal (uno por uno)
- *args → Tupla
- **kwargs → Diccionario

REGLA PARA RECORDAR

Parámetros normales	→	uno por uno
*args	→	recibe todos los argumentos posicionales restantes (TUPLA)
**kwargs	→	recibe todos los argumentos con nombre (DICCIONARIO)

Ejemplo práctico

```python
def crear_usuario(nombre, *telefonos, **datos):
    print(f"Usuario: {nombre}")

    print("\nTeléfonos:")
    for t in telefonos:
        print("-", t)

    print("\nDatos adicionales:")
    for clave, valor in datos.items():
        print(f"{clave}: {valor}")

crear_usuario(
    "Juan",
    "11-5555-1111",
    "11-4444-2222",
    edad=30,
    ciudad="Córdoba",
    profesion="Data Scientist"
)
```

Idea visual

```
crear_usuario(
  "Juan",
  "11-5555",
  "11-4444",
  edad=30,
  ciudad="Córdoba"
)
```

- nombre → "Juan"
- *args → ("11-5555", "11-4444")
- **kwargs → { "edad": 30, "ciudad": "Córdoba" }

MNEMOTECNIA

*args → A argumentos por posición → TUPLA ( )

**kwargs → K ey W ord Arg umentos → DICCIONARIO { }

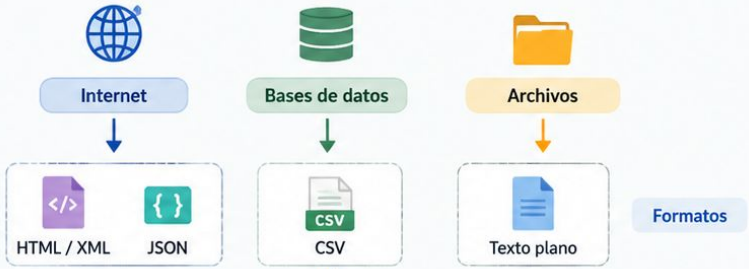
¿Qué es Pandas?

La biblioteca de Python para analizar datos tabulares

Pandas está construido sobre NumPy y se especializa en el **análisis de datos** más complejos.

- Puede trabajar con **diferentes tipos de datos** en una misma tabla.
- Permite **organizar, limpiar, transformar** y **analizar** datos de forma eficiente.
- Es una herramienta esencial para científicos de datos.

¿De dónde vienen los datos?



NumPy vs Pandas

NumPy

Trabaja con **arrays numéricos homogéneos.**

[1.2 3.4 5.6 7.8]

Pandas

Trabaja con **datos heterogéneos y tabulares.**

Nombre	Edad	Ciudad
Ana	22	Lima
Luis	35	Bogotá
Sofia	18	Quito
...	...	...

Objetivo de Pandas: Organizar y estructurar los datos para que puedan ser **entendidos** por nosotros y por las computadoras.

Pandas es la herramienta óptima para **recopilar, organizar y analizar** datos tabulares. Es esencial en casi todo el flujo de trabajo de Ciencia de Datos.

Series y DataFrames

Las dos estructuras fundamentales de pandas

SERIES (1 dimensión)

Una **columna** de datos.

	Edad
0	22
1	35
2	18
3	41
4	29

- **Nombre** (label)
- **Índice** (index)
- **Valores** (values)

Puede contener:

- números
- texto
- fechas
- booleanos

```
s = pd.Series([22, 35, 18, 41, 29], name='Edad')
```

DATAFRAME (2 dimensiones)

Conjunto de múltiples **Series** (columnas).

Columnas

	Nombre	Edad	Ciudad
0	Ana	22	Lima
1	Luis	35	Bogotá
2	Sofía	18	Quito

- Nombres de columnas (labels)
- Filas (observaciones)

```
df = pd.DataFrame({
    'Nombre': ['Ana', 'Luis', 'Sofía'],
    'Edad': [22, 35, 18],
    'Ciudad': ['Lima', 'Bogotá', 'Quito']
})
```

Cada columna del DataFrame es una **Series**.

RELACIÓN ENTRE ESTRUCTURAS

Escalar
Un solo valor
(ej: 5, 'texto')

→

Series (1D)
Conjunto de valores con un índice

→

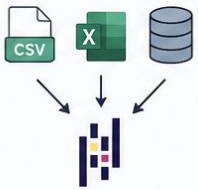
DataFrame (2D)
Conjunto de Series alineadas por índice

¿Por qué Pandas es tan importante?

Pandas permite organizar, explorar, limpiar y combinar datos para convertirlos en información útil.

pandas

1. LEER DATOS



- CSV
- Excel
- Bases de datos (SQL)
- JSON y otros formatos

2. EXPLORAR DATOS



- Primeras filas (head)
- Últimas filas (tail)
- Tipos de datos (info)
- Columnas e índices

3. LIMPIAR DATOS



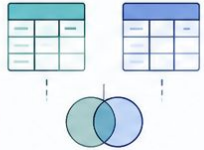
- Eliminar filas / columnas
- Cambiar nombres
- Reemplazar valores
- Manejar valores faltantes (fillna)

4. ANALIZAR DATOS



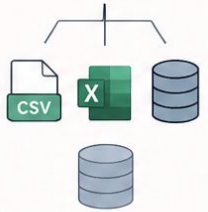
- Estadísticas descriptivas
- Filtros y selección
- Agrupaciones (groupby)
- Operaciones entre columnas

5. COMBINAR INFORMACIÓN



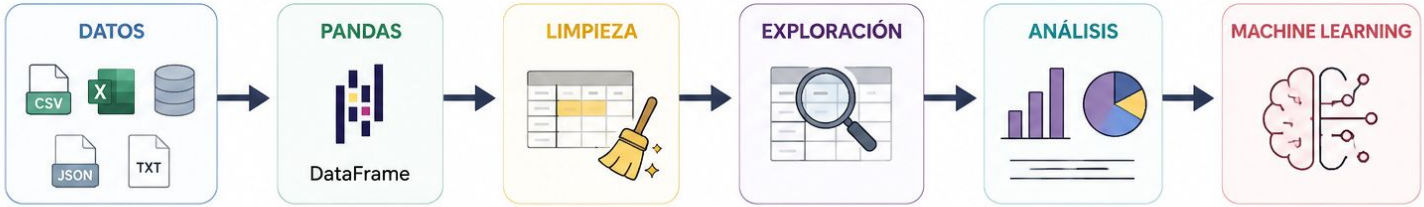
- Merge (unir por claves)
- Join (unir por índices)
- Concat (apilar tablas)

6. EXPORTAR / USAR



- Guardar resultados
- Visualizar
- Usar en Machine Learning

FLUJO TÍPICO CON PANDAS



Pandas es la herramienta central para transformar datos crudos en información estructurada y lista para ser analizada.

SQL vs Pandas

No compiten, se complementan en el análisis de datos

SQL
(Lenguaje de consultas)

- Vive dentro de una base de datos.
- Usa tablas persistentes almacenadas.
- Optimizado para consultar y filtrar millones de registros.
- Procesa datos donde están almacenados (en disco).
- Lenguaje declarativo: le dices QUÉ quieres obtener.

Ideal para:
consultas, filtrado y extracción de grandes volúmenes de datos.

EJEMPLO SQL

```
SELECT nombre, ventas, ciudad
FROM clientes
WHERE ciudad = 'Buenos Aires'
ORDER BY ventas DESC
LIMIT 10;
```

nombre	ventas	ciudad
Ana	1200	Buenos Aires
Luis	980	Buenos Aires
María	750	Buenos Aires
...	...	...

FLUJO TÍPICO

1. DATOS ALMACENADOS



Base de Datos

2. CONSULTA CON SQL

```
SQL
SELECT *
FROM ventas
WHERE fecha >= '2024-01-01';
```

3. CARGA A PANDAS

```
import pandas as pd
df = pd.read_sql(query, conexion)
```

4. ANÁLISIS Y TRANSFORMACIÓN CON PANDAS

pandas

fecha	producto	ventas	ciudad	ganancia
2024-01-01	TV	1200	BA	300
2024-01-01	Celular	980	BA	250
2024-01-01	Notebook	750	BA	180
...	...	...	...	...

- Limpieza de datos (filtrar, eliminar nulos, renombrar, etc.)
- Análisis y estadísticas (promedios, agrupaciones, correlaciones, etc.)
- Preparación para Machine Learning (modelos predictivos)

Pandas
(Biblioteca de Python)

- Vive en la memoria de Python.
- Usa DataFrames (estructuras temporales).
- Optimizado para análisis y manipulación de datos.
- Procesa datos en memoria.
- Es una biblioteca de Python: le dices CÓMO hacerlo.

Ideal para:
limpieza, transformación, análisis exploratorio y Machine Learning.

EJEMPLO PANDAS

```
# crear nueva columna                                  Crear nuevas columnas
df['ganancia'] = df['ventas'] - df['costo']

# filtrar                                              Filtrar filas
df_ba = df[df['ciudad'] == 'Buenos Aires']

# agrupar y obtener promedio                           Agrupar y calcular
df_ba.groupby('producto'))['ganancia'].mean()

# ordenar                                              Ordenar
df_ordenado = df.sort_values('ventas', ascending=False)
```

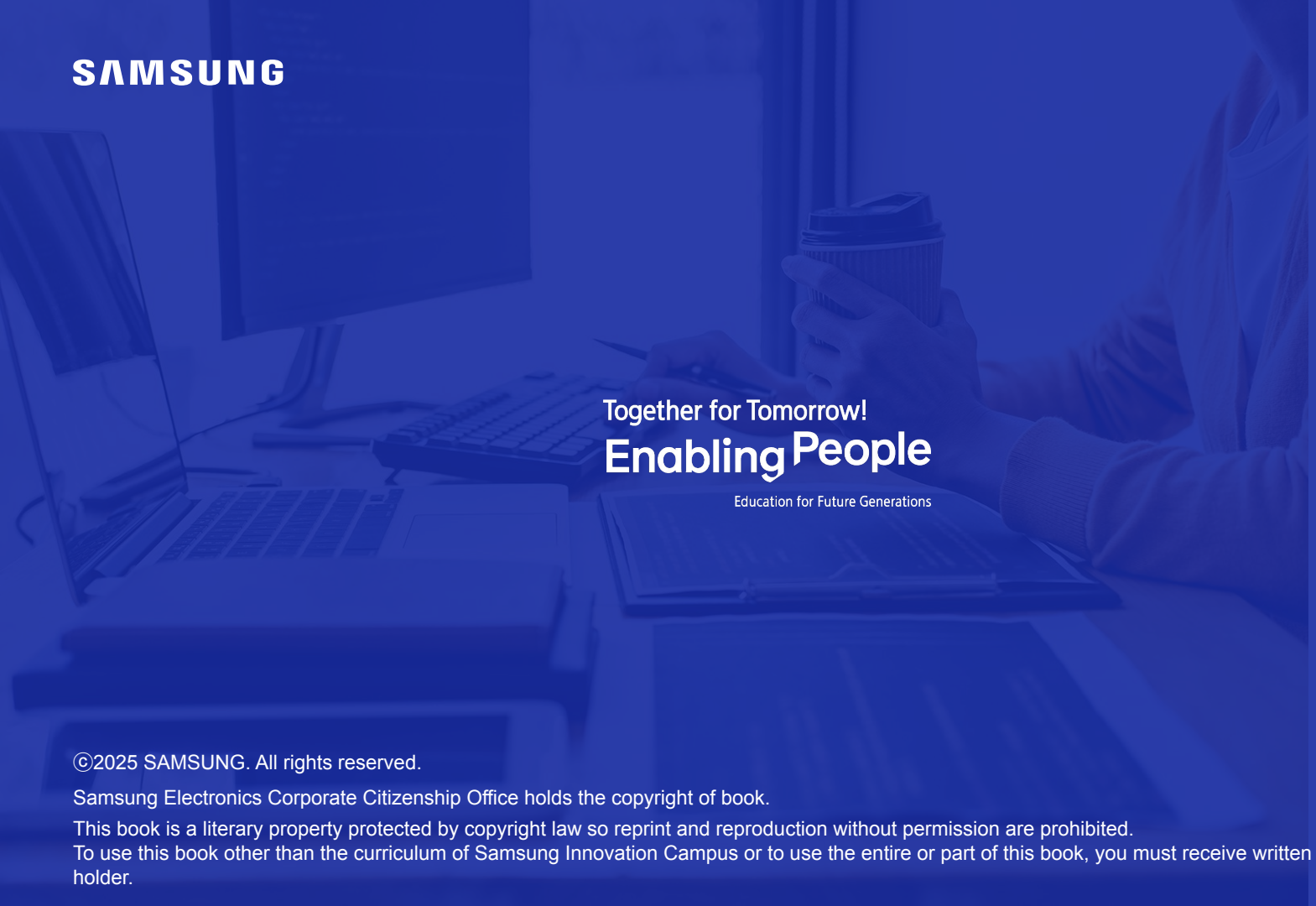
EQUIVALENCIAS COMUNES

SELECT	WHERE	ORDER BY	GROUP BY	JOIN	COUNT	AVG	MAX / MIN
df[['col1', 'col2']]	df[df['condicion']]	df.sort_values('col')	df.groupby('col')	df.merge(otro_df, on='id')	df['col'].count()	df['col'].mean()	df['col'].max() / .min()

IDEA CLAVE

SQL te ayuda a obtener los datos que necesitas. Pandas te ayuda a convertir esos datos en información útil.

Juntos forman un flujo poderoso para el análisis de datos.



ⓒ2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.